

GENEL PROJE ÖNİZLEMESİ: Zekâ-i Masnû

Eren Ergin
Hüseyin Berat Özen
Ali Kemal Yavuz
Atıf Kışla

<https://github.com/ErenErgin78/Zekai-Masnu-UMAY.git>

1. GİRİŞ

1.1. Projenin Amacı

Bu proje, UMay (Uzman Mahsül Analiz Yardımcısı) çiftçiler ve tarım profesyonelleri için geliştirilmiş yapay zeka tabanlı bir karar destek sistemi sunmayı amaçlamaktadır. Projenin temel hedefi, kullanıcılara tarımla ilgili eyleme geçirilebilir, somut ve veriye dayalı öneriler sunmaktır. , bu amacı gerçekleştirmek için dört ana işlevsel alanı tek bir entegre platformda birleştirmektedir:

- Gerçek Zamanlı Toprak Analizi:** Kullanıcının manuel olarak girdiği veya otomatik olarak tespit edilen koordinatlara dayanarak, HWSD2 (Harmonized World Soil Database) veritabanından detaylı toprak özellikleri (pH, organik karbon, doku, su kapasitesi vb.) analizi sunar.
- Detaylı Hava Durumu Takibi:** Open-Meteo API'lerini kullanarak, anlık, saatlik, günlük (16 güne kadar) ve geçmişe dönük tarihsel hava durumu verilerini (yağış, sıcaklık, nem, rüzgar vb.) sağlar.
- Makine Öğrenmesi ile Ürün Önerisi:** Toprak analizi verileri ile iklim verilerini birleştiren eğitilmiş bir makine öğrenmesi modelini (model.pkl) kullanarak, mevcut arazi için en uygun bitki önerilerini olasılık ve güven skorlarıyla birlikte sunar. Ayrıca bu veri seti bu projeyi hazırlayanlar tarafından sıfırdan gerçek toprak verilerine ve iklim verilerine dayanılarak hazırlanmıştır.
- Organik Tarım Bilgi Bankası (RAG):** Organik tarım, gübreleme, haşere kontrolü ve sertifikasyon gibi konularda, veri kazıyıcılar (Scraper) ile toplanan ve vektör veritabanına (ChromaDB) işlenen yaklaşık 300 PDF ve doküman üzerinden yapay zeka destekli (Retrieval-Augmented Generation) cevaplar üretir.

1.2. Projenin Vizyonu

UMAY projesinin vizyonu, Türk çiftçisine yönelik, tarımsal veri ihtiyaçlarını karşılayan merkezi ve akıllı bir "uzman asistan" olmaktır. Sadece ham veri sunmanın ötesinde, bu verileri yorumlayabilen, birleştirebilen ve anlamlı sonuçlar üretebilen bir sistem hedeflenmiştir.

Bu vizyon, projenin modüler mimarisi ile desteklenmektedir:

- **Entegre Zeka (LLM):** Sistemin "beyni" olarak görev yapan Gemini tabanlı dil modeli (LLM), kullanıcının talebini analiz eder.
- **Çoklu Araç Kullanımı (Multi-Tool):** LLM, basit bir soruyu (örn: "Hava nasıl?") cevaplamak için tek bir aracı/toolu (get_weather) çalıştırırken; karmaşık bir soruyu (örn: "Bu bölgenin toprak ve iklim yapısına göre hangi organik ürünü ekmeliyim?") cevaplamak için birden fazla aracı/toolu (analyze_soil, get_weather, ml, rag_tool) akıllı bir zincir (chain) veya ajan (agent) mantığını kullanarak açıklar.
- **Sürdürülebilirlik ve Verimlilik:** RAG sistemi aracılığıyla organik ve sürdürülebilir tarım pratiklerini teşvik etmek ve ML modeli ile en verimli ürün seçimini sağlayarak tarımsal operasyonları optimize etmek, projenin uzun vadeli vizyonunun temelini oluşturur.

2. SİSTEM MİMARİSİ VE PROJE KATMANLARI

UMAY projesi, modern ve modüler bir yazılım mimarisi üzerine inşa edilmiştir. Sistem, bağımsız ancak entegre çalışabilen dört ana katmandan oluşmaktadır: API'lar, RAG (Bilgi Bankası), Yapay Zeka (LLM) ve Frontend (Kullanıcı Arayüzü).

Bu katmanlı yapı, sistemin esnekliğini, ölçeklenebilirliğini ve bakım kolaylığını artırmaktadır.

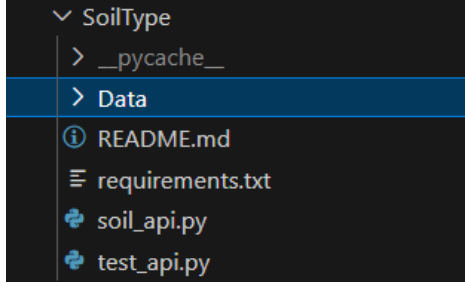
Veri akışı şu şekilde özetlenebilir: **Kullanıcı → Frontend → LLM Katmanı → [Backend API / RAG] → LLM Katmanı → Frontend → Kullanıcı.**

2.1. Backend API Katmanı (Veri ve İş Mantığı)

Projenin iş mantığının ve temel veri servislerinin yer aldığı bu katman, Python tabanlı **FastAPI** web framework'ü üzerine kurulmuştur. Bu katman, LLM katmanının "araçları" (tools) tarafından çağrılmak üzere tasarlanmış bir mikroservis mimarisi sunar.

main.py dosyası, tüm alt servisleri (router) tek bir ana uygulama altında birleştiren (port 8000 üzerinde çalışan) giriş noktasıdır. Bu katmanın sağladığı temel servisler şunlardır:

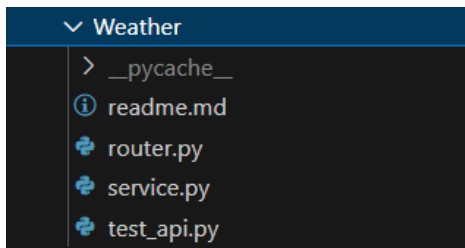
2.1.1. Toprak Analizi Servisi (SoilType)



Bu servis, projenin en kritik veri sağlayıcılarından biridir ve soil_api.py dosyasında tanımlanmıştır.

- **Amaç:** Belirtilen coğrafi koordinatlara ait detaylı toprak özelliklerini sağlamak.
- **Çalışma Mantığı:**
 1. **Konum Tespiti:** Soil API ile ilgili Endpoint'ler, hem manuel (/analyze) hem de otomatik GPS tabanlı (/analyze/auto) konum tespiti yöntemlerini destekler.
 2. **Raster Veri Okuma:** Gelen (enlem, boylam) koordinatları kullanarak, rasterio kütüphanesi aracılığıyla HWSD2.bil (Harmonized World Soil Database raster haritası) dosyası okunur. Bu okuma işlemi, o koordinata denk gelen piksel değerini, yani benzersiz **Toprak ID (Soil ID)** değerini döndürür.
 3. **Veritabanı Sorgusu:** Elde edilen Soil ID kullanılarak, pyodbc kütüphanesi aracılığıyla HWSD2.mdb (Microsoft Access veritabanı) dosyasına bir SQL sorgusu gönderilir.
 4. **Veri Döndürme:** Bu sorgu, o ID'ye ait tüm detaylı katman verilerini (örn: pH, Organik Karbon, Kil/Silt/Kum oranları, Su kapasitesi vb.) çeker ve bu verileri JSON formatında LLM'e (veya doğrudan Frontend panelinine) sunar.

2.1.2. Hava Durumu Servisi (Weather)



Bu servis, router.py (Weather dizinindeki) dosyası aracılığıyla sağlanır ve harici bir hava durumu sağlayıcısı için modern bir arayüz (wrapper) görevi görür.

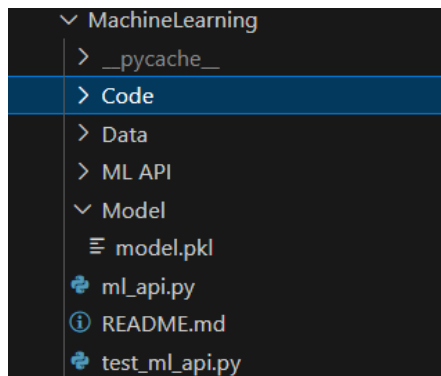
- **Amaç:** Anlık, tahminî ve geçmişe dönük meteorolojik verileri sağlamak.
- **Çalışma Mantığı:**
 1. **Teknoloji:** Harici Open-Meteo API servisine httpx (veya requests) kütüphanesi ile güvenli API istekleri atar.
 2. **Kapsamlı Veri:** Sadece basit sıcaklık bilgisi değil; saatlik (/hourlyweather), günlük (/dailyweather) ve geçmişe dönük (/bydate) olmak üzere; yağış olasılığı, toprak nemi, rüzgar hızı, güneşlenme süresi gibi tarımsal açıdan kritik birçok parametreyi sağlar.
 3. **WMO Kod Çevirisi:** API'den gelen standart WMO (Dünya Meteoroloji Örgütü) hava durumu kodlarını (örn: 61, 80) kullanıcıların anlayabileceği Türkçe metinlere (örn: "Yağmur (Hafif)", "Sağanak Yağmur (Hafif)") çevirir.

2.1.3. Makine Öğrenmesi Servisi (MachineLearning)

Bu rapor, çeşitli coğrafi konumlara ait toprak analizi, iklim verileri ve diğer bölgesel özellikleri kullanarak, bu konumlarda yetiştirilmeye uygun olan tarımsal ürünleri tahmin etmeyi amaçlayan makine öğrenmesi projesinin detaylı analizini sunmaktadır. Proje, 38 farklı tarım ürününü kapsayan bir **çok etiketli sınıflandırma (multi-label classification)** problemi olarak ele alınmıştır.

Amaç, veri hazırlama sürecinden model optimizasyonuna ve nihai performans değerlendirmesine kadar tüm adımları belgelemek ve en başarılı tahmin modelini belirlemektir.

2.1.3.2. Veri Hazırlama Süreci (Verinin Yolculuğu)



Modellerin eğitiminde kullanılan veriler, ham halinden analiz edilebilir bir formata gelene kadar birkaç kritik aşamadan geçirilmiştir:

1. **Veri Zenginleştirme:** İlk olarak, ham veri setindeki eksik meteorolojik (hava durumu) veriler tespit edilmiştir. Bu eksiklikler, coğrafi koordinatlar (enlem ve boylam) kullanılarak Haversine mesafe analizine tabi tutulmuş ve her eksik satır, kendisine coğrafi olarak en yakın komşu lokasyonun verileriyle doldurulmuştur.

2. **Özellik Azaltma ve Korelasyon Analizi:** Veri setindeki 36 özellik (feature) arasında yüksek korelasyona sahip (korelasyon eşiği > 0.85) olan sütunlar incelenmiştir. Modeli yanıltabilecek ve gereksiz karmaşıklık yaratabilecek bu yüksek korelasyonlu özellikler (örneğin, "Ortalama Sıcaklık" ile "Ortalama En Yüksek Sıcaklık" gibi) stratejik olarak veri setinden çıkarılarak boyut azaltılmıştır.
3. **Çok Etiketli (Multi-Label) Dönüşüm:** Bu, projenin en kritik veri hazırlama adımıdır. Analiz, "Binary Relevance" (İkili Uygunluk) adı verilen çok etiketli bir sınıflandırma tekniğine dayanmaktadır. Bu yaklaşım için veri seti dönüştürülmüştür. Her bir lokasyon (satır) için, 38 farklı ürünün tamamı ayrı birer sütun haline getirilmiştir (örn: label_Arpa, label_Buğday, label_Pamuk...). Eğer o lokasyonda ilgili ürün yetişiyorsa "1", yetişmiyorsa "0" değeri atanmıştır.

Sonuç olarak, her biri 36 özelliğe sahip 1564 örnek satırdan ve bu satırların her biri için 38 adet ikili (0/1) etiketten oluşan nihai analiz veri seti elde edilmiştir.

2.1.3.3.1. Veri Bölütleme ve Ölçeklendirme

Modelin performansını objektif olarak ölçebilmek için veri seti %80 eğitim (1251 örnek) ve %20 test (313 örnek) olarak ikiye ayrılmıştır.

Modellerin (özellikle SVM ve Logistic Regression gibi) farklı ölçeklerdeki özelliklerden olumsuz etkilenmemesi için tüm 36 özellik sütunu, **StandardScaler** yöntemi kullanılarak standartlaştırılmıştır. Bu işlem, her bir özelliğin ortalamasını 0 ve standart sapmasını 1 yapacak şekilde veriyi yeniden ölçeklendirmiştir.

2.1.3.3.2. Model Yaklaşımı ve Optimizasyon

Bu problem, her bir ürün için ayrı birer model eğitilmesini gerektiren MultiOutputClassifier (Çok Çıktılı Sınıflayıcı) yaklaşımıyla çözülmüştür. Bu strateji, 38 farklı ürün etiketi için 38 adet bağımsız sınıflandırma modeli eğitir.

Aşağıdaki 8 farklı makine öğrenmesi algoritması test edilmiştir:

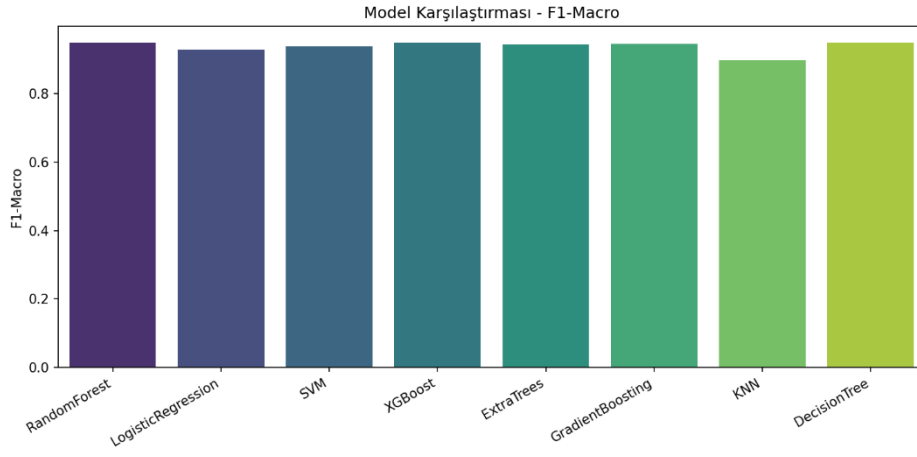
- RandomForest
- XGBoost
- DecisionTree
- GradientBoosting
- ExtraTrees
- SVM (Support Vector Machine)
- LogisticRegression
- KNN (K-Nearest Neighbors)

Her bir model için en iyi hiperparametre setini bulmak amacıyla GridSearchCV (Kapsamlı Parametre Arama) tekniği kullanılmıştır. Optimizasyon metriği olarak, özellikle etiketlerin veri setindeki dağılımının dengesiz olabildiği (bazı ürünlerin az, bazılarının çok bulunması) durumlara karşı daha dayanıklı olan **F1-Macro** skoru seçilmiştir.

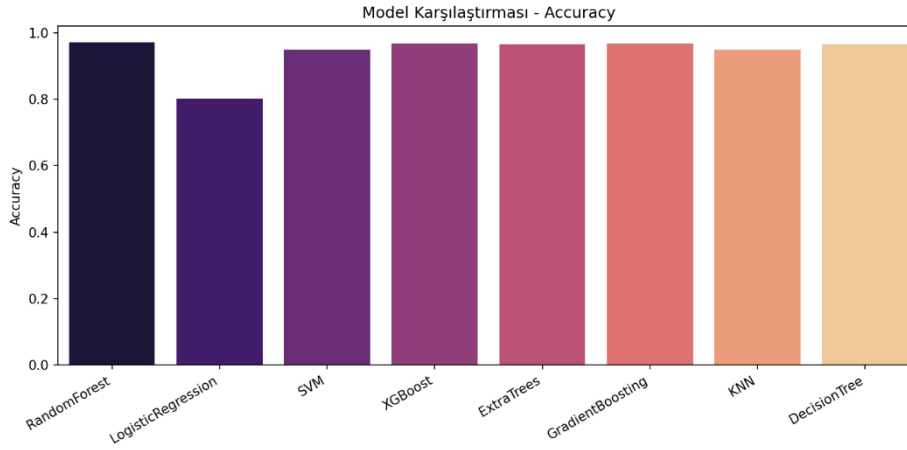
2.1.3.4. Model Karşılaştırması ve Sonuçlar

Sekiz farklı algoritmanın test seti üzerindeki performansları F1-Macro ve Doğruluk (Accuracy) metriklerine göre karşılaştırılmıştır.

Aşağıdaki grafik, modellerin F1-Macro skorlarını göstermektedir. Bu metrik, modelin etiket bazlı performansını (hem Precision hem de Recall) dengeli bir şekilde ölçer.



İkinci grafik, modellerin genel doğruluk (Accuracy) oranlarını, yani tüm etiketler üzerinden doğru yapılan tahminlerin toplam oranını göstermektedir.



2.1.3.4.1. Sonuçların Değerlendirilmesi

Aşağıdaki tablo, modellerin test verisi üzerindeki performanslarını özetlemektedir:

Sıra	Model	F1-Macro Skoru	Doğruluk (Accuracy)	Hamming Loss
1	RandomForest	0.9487	0.9712	0.0026
2	XGBoost	0.9486	0.9681	0.0026
3	DecisionTree	0.9479	0.9649	0.0028
4	GradientBoosting	0.9439	0.9681	0.0028
5	ExtraTrees	0.9428	0.9649	0.0030
6	SVM	0.9380	0.9489	0.0035
7	LogisticRegression	0.9286	0.8019	0.0092
8	KNN	0.8969	0.9489	0.0056

En İyi Model: RandomForest

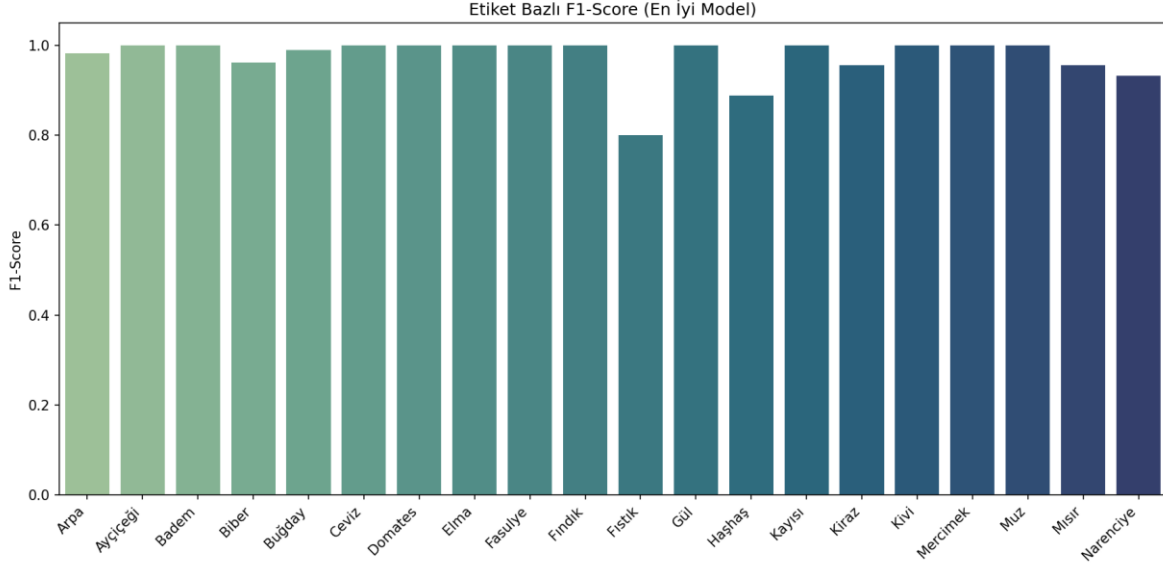
- En İyi F1-Macro: 0.9487
- En İyi Doğruluk: 0.9712

Değerlendirme sonucunda, hem F1-Macro hem de Doğruluk skorlarında en yüksek başarıyı gösteren **RandomForest** modeli, bu problem için **en iyi model** olarak seçilmiştir. Hamming Loss (hatalı tahmin edilen etiketlerin oranı) metriğinde de 0.0026 ile en düşük hata oranlarından birine sahiptir.

2.1.3.5. En İyi Modelin Derinlemesine Analizi (RandomForest)

En iyi model olarak seçilen RandomForest'un performansı, 38 ürünün her biri için etiket bazlı olarak incelenmiştir.

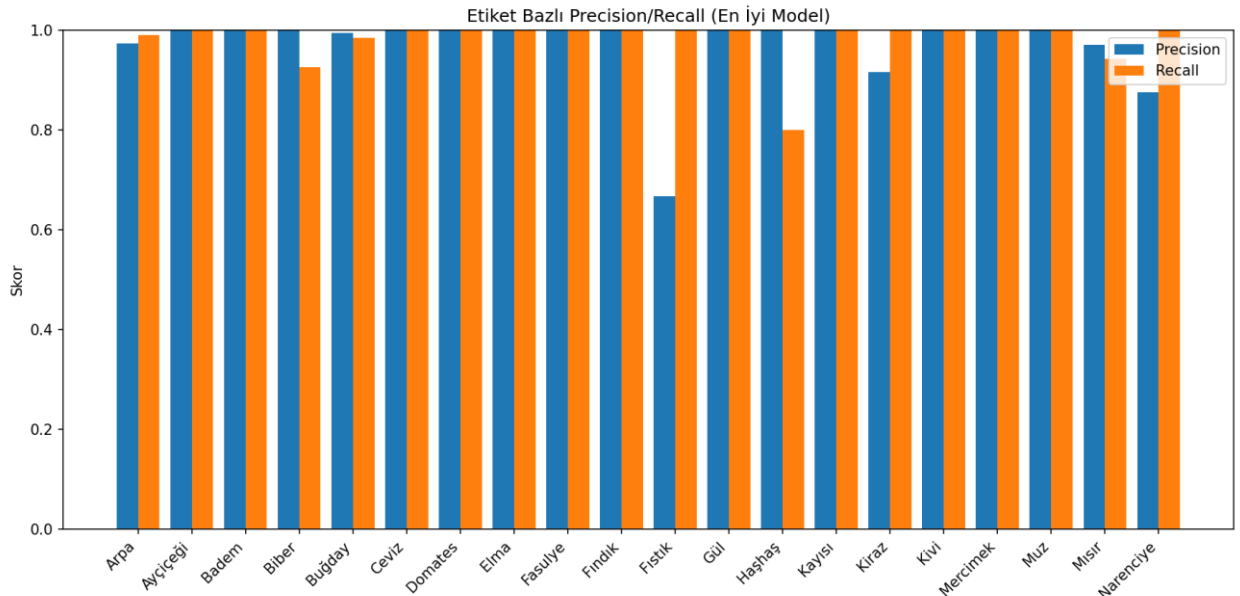
Aşağıdaki grafik, modelin en sık rastlanan 20 ürün için F1-Skor performansını göstermektedir.



Grafikten de görüleceği üzere, model "Ayçiçeği", "Badem", "Ceviz", "Domates", "Elma", "Fasulye", "Fındık", "Kayısı", "Kivi", "Mercimek", "Muz", "Nohut", "Patlıcan" ve daha birçok ürünü **1.0 F1-Skor** ile, yani **%100 hatasız** bir şekilde sınıflandırmıştır.

"Arpa" (0.9820), "Buğday" (0.9900) ve "Mısır" (0.9565) gibi ürünlerde de performansı mükemmele yakındır.

Modelin zorlandığı (daha düşük F1 skorlu) etiketler "Fıstık" (0.8000) ve "Haşhaş" (0.8889) olmuştur. Bu durumun daha iyi anlaşılması için Precision (Kesinlik) ve Recall (Duyarlılık) skorlarına bakılmıştır.



Bu grafik, modelin yaptığı hataların türünü göstermektedir:

- **Fıstık (F1: 0.8000):** Modelin Recall skoru 1.0 (tüm "Fıstık" olan yerleri bulmuş), ancak Precision skoru 0.6667'dir. Bu, modelin "Fıstık" olmayan bir yeri 1 kez hatalı olarak "Fıstık" şeklinde etiketlediği (False Positive) anlamına gelir.
- **Haşhaş (F1: 0.8889):** Modelin Precision skoru 1.0 (tüm "Haşhaş" tahminleri doğru), ancak Recall skoru 0.8000'dir. Bu, modelin "Haşhaş" olan bir lokasyonu 1 kez gözden kaçırdığı (False Negative) anlamına gelir.

Raporda F1-Skoru 0.0 çıkan "Yer Fıstığı" gibi veya düşük F1-Skorlu "Şerbetçi Otu" gibi etiketlerin performansı, bu etiketlerin veri setinde çok nadir bulunmasından (aşırı dengesizlik) kaynaklanmaktadır ve bu durum, modelin genel başarısını (0.9487 F1-Macro) gölgelemektedir.

2.1.3.6. Genel Sonuç

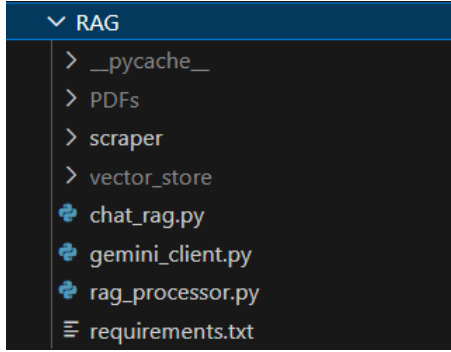
Uygulanan veri hazırlama pipeline'ı ve kapsamlı model optimizasyonu sonucunda, **RandomForest** sınıflandırıcısı %97.12 genel doğruluk ve %94.87 F1-Macro skoru ile tarımsal ürün uygunluğunu tahmin etmede üstün bir başarı sergilemiştir. Model, 38 ürünün büyük çoğunluğunu mükemmel yakın bir performansla tespit edebilmektedir.

2.1.4. Kullanım (API Çalışma Akışı)

Servisin `ml_api.py` dosyasında tanımlanan `POST /ml/analyze` endpoint'i, sistemin ana ürün önerme işlevini yerine getirir. Bu uç nokta çağrıldığında, `MLService` sınıfı aşağıdaki adımları sırayla yürütür:

1. **Toprak Verisi Çekme:** İlk olarak, `SoilType API'sine` (<http://localhost:8000/soiltype>) bir istek atılır. Kullanıcının `Auto` (otomatik) veya `Manual` (belirli koordinatlarla) metoduna göre ilgili lokasyonun detaylı toprak özellikleri (örn: kil, kum, organik karbon, WRB kodu vb.) alınır.
2. **İklim Verisi Ekleme:** Toprak verilerine ek olarak, modelin ihtiyaç duyduğu iklim özellikleri (örn: aylık yağış, sıcaklık ortalamaları) `Data/final5.csv` dosyasındaki özet verilerden bulunur. Mevcut sistemde bu, koordinatlara en yakın veriyi bularak veya temsili bir satır seçerek yapılır.
3. **Özellik Vektörü Oluşturma:** Toprak ve iklimden gelen bu iki farklı veri seti, `_prepare_vector` fonksiyonu aracılığıyla birleştirilir. Modelin eğitildiği 36 özelliğin tamamını içeren tek bir özellik vektörü (Numpy dizisi) oluşturulur.
4. **Ölçeklendirme:** Oluşturulan bu birleşik vektör, modelle birlikte kaydedilmiş olan `StandardScaler` kullanılarak ölçeklendirilir. Bu, verinin modelin eğitildiği formata (aynı ortalama ve standart sapmaya) getirilmesini sağlar.
5. **Tahmin Yapma:** Ölçeklenmiş vektör, `Model/model.pkl` dosyasından yüklenen (RandomForest) makine öğrenmesi modeline (`_predict` fonksiyonu) beslenir. Model, 38 ürünün her biri için bir olasılık skoru üretir.
6. **JSON Yanıtı Döndürme:** En yüksek olasılığa sahip ürünler (örn: ilk 5) `PlantRecommendation` nesneleri olarak listelenir ve tüm analiz süreci (kullanılan koordinatlar, toprak verileri, iklim verileri ve öneriler) `MLAnalysisResponse` Pydantic modeli aracılığıyla yapılandırılmış bir JSON formatında çağrıya yanıt olarak döndürülür.

2.2. Bilgi Bankası Katmanı (RAG - Retrieval-Augmented Generation)



RAG katmanı sistemin bilgi hafızası olarak görev yapar. Bu katmanın temel amacı, "Organik tarım nedir?", "Kompost nasıl yapılır?" veya "Domates zararlıları ile nasıl mücadele edilir?" gibi spesifik ve bilgiye dayalı soruları, projenin kendi özel doküman kütüphanesinden bularak cevaplamaktır.

Bu sistem, LLM'in genel internet bilgisine dayanarak cevap vermesinin önüne geçer ve sadece sisteminize yüklenen güvenilir kaynaklardan (PDF'ler, makaleler, rehberler) beslenen, doğruluğu yüksek yanıtlar üretilmesini sağlar.

Bu katmanın çalışma mimarisi üç ana adımdan oluşur: Veri İşleme, Veri Depolama ve Veri Geri Getirme.

2.2.1. Veri İşleme ve Vektörleştirme (Processing & Vectorization)

Bu aşama, rag_processor.py dosyası tarafından yönetilen "Akıllı Doküman Yönetimi" sürecidir.

- Veri Toplama:** PDFs klasörüne eklenen tüm dokümanlar (PDF, TXT, DOCX) sistem tarafından taranır. Bu klasör, projeye harici veri kazıyıcılar tarafından toplanan akademik makaleler ve tarım rehberleri ile doldurulur. Hazırlamış olduğumuz projede yaklaşık 300 makale, doküman, yazı kullanılmıştır ve bu kaynakların tamamı web scraper yöntemi ile elde edilmiştir.
- Akıllı Yükleme (Robust Loading):** Sistem, bir dokümanı okumak için çok adımlı bir deneme mekanizması kullanır. PDF'ler için öncelikle en hızlı ve güvenilir olan PyMuPDF (fitz) kütüphanesini dener. Başarısız olursa, LangChain kütüphanesinin kendi PyPDFLoader veya UnstructuredWordDocumentLoader gibi alternatif pdf işleme yöntemlerine geçer.
- Akıllı Senkronizasyon:** rag_processor.py, vector_store içinde hangi dosyaların zaten işlenmiş olduğunu bilir. Yeni bir işlem başlatıldığında, sadece PDFs klasörüne **yeni eklenen** dosyaları işler ve klasörden **silinmiş** olan dosyaların vektörlerini veritabanından temizler.
- Parçalama (Chunking):** Yüklenen uzun dokümanlar, RecursiveCharacterTextSplitter kullanılarak 1000 karakterlik, birbiriyle 200 karakter örtüşen (overlap) daha küçük metin parçalarına (chunks) bölünür.

5. **Gömme (Embedding):** Tüm bu metin parçaları, HuggingFaceEmbeddings (özellikle sentence-transformers/all-MiniLM-L6-v2 modeli) kullanılarak anlamsal olarak anlaşılabilen matematiksel vektörlere dönüştürülür.

2.2.2. Veri Depolama (Vektör Veritabanı)

Oluşturulan bu anlamsal vektörler, kalıcı bir depolama çözümüne ihtiyaç duyar.

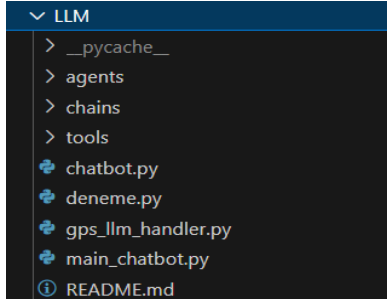
- **Teknoloji:** Proje, bu vektörleri depolamak için açık kaynaklı bir vektör veritabanı olan **ChromaDB**'yi kullanır.
- **Depolama:** ChromaDB, tüm bu vektörleri ve kaynak meta verilerini vector_store adlı bir klasörde fiziksel olarak saklar. Bu, sistem her yeniden başladığında tüm PDF'leri tekrar işlemesine gerek kalmadan bilgi hafızasına anında erişebilmesini sağlar.

2.2.3. Veri Geri Getirme ve Cevap Üretme (RAG Akışı)

Kullanıcı bir soru sorduğunda, chat_rag.py ve rag_tool.py tarafından yönetilen RAG akışı tetiklenir:

1. **Sorgu:** Kullanıcının sorusu ("Organik gübre nasıl yapılır?") alınır.
2. **Benzerlik Araması (Search):** Bu soru da (2.2.1'deki modelle) bir vektöre dönüştürülür ve ChromaDB veritabanında "anlamsal olarak en çok benzeyen" metin parçalarını (k=3 adet) bulmak için bir benzerlik araması yapılır.
3. **Bağlam Oluşturma (Retrieve):** Veritabanından dönen bu 3 adet alakalı metin parçası ("context") birleştirilir.
4. **Cevap Üretme (Generate):** Bu "context" ve kullanıcının orijinal sorusu, gemini_client.py aracılığıyla Google Gemini LLM'e gönderilir. LLM'e şu talimat verilir: "Bu soruya, *sadece sana verdiğim bu bağlamı (context) kullanarak* cevap ver".
5. **Sonuç:** LLM, internetten değil, sadece sizin sağladığınız PDF'lerden aldığı bilgiyle zenginleştirilmiş, kaynaklara dayalı bir cevap üretir ve kullanıcıya sunar

2.3. Yapay Zeka Katmanı (LLM – Language Model Layer)



Bu katman, projenin merkezi sinir sistemi ve "orkestratörü" olarak görev yapar. Kullanıcı ile Backend servisleri (API, RAG) arasındaki akıllı arayüzdür. Sistemin temel teknolojisi, Google'ın **Gemini** (gemini-2.5-flash modeli) dil modeline dayanmaktadır.

Bu katmanın rolü, basit bir soru-cevap botu olmanın çok ötesindedir. UMay, bir "Akıllı Ajan" (Intelligent Agent) olarak tasarlanmıştır. chatbot.py dosyasında tanımlanan sistem, gelen doğal dil sorgusunu analiz eder ve bu sorguyu çözmek için hangi araca (tool), araç zincirine (chain) veya alt-ajana (agent) başvurması gerektiğine kendisi karar verir.

Bu karar verme süreci, chatbot.py içinde tanımlanan ve Gemini modeline "Function Calling" (Fonksiyon Çağırma) yeteneği olarak sunulan araç setine dayanır.

2.3.1. Ana Orkestratör (Sistem Yöneticisi ve Chatbot)

Sistemin ana orkestrasyonu iki temel dosya tarafından yönetilir: main_chatbot.py (Sistem Yöneticisi) ve chatbot.py (Konuşma Yöneticisi/Ajan).

1. main_chatbot.py (Sistem ve API Yöneticisi) Bu dosya, tüm projenin ana giriş noktası (entry point) olarak görev yapar. Roller şunlardır:

- **Sunucu Başlatma:** Bir **FastAPI** sunucusu (Port 8001) başlatarak Frontend (index.html) ile Backend (LLM) arasındaki iletişimi sağlar.
- **Servis Yönetimi:** Projenin çalışması için kritik olan bağımsız Backend API servislerini (Soil+Weather API'sini Port 8000'de ve ML API'sini Port 8003'te) alt süreçler (subprocess) olarak başlatır, sağlık durumlarını kontrol eder ve yönetir.
- **Önyükleme (Initialization):** Tüm servisleri (araçları) koordine eden UMayServiceManager sınıfını (main.py'dan) başlatır.
- **İstek Yönlendirme:** OrganicFarmingChatBot sınıfından (chatbot.py) bir örnek (instance) oluşturur ve Frontend'den gelen /chat/ HTTP isteklerini bu "beyin" örneğine iletir.

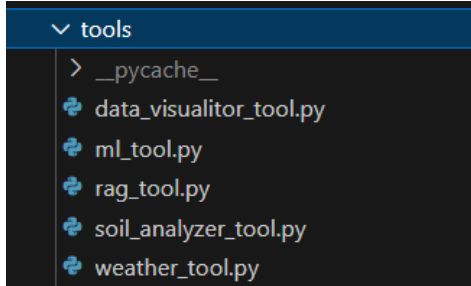
2. chatbot.py (Konuşma Yöneticisi ve Ajan) Bu dosya, OrganicFarmingChatBot sınıfını tanımlar ve asıl "beyin" olarak çalışır. main_chatbot.py tarafından başlatıldıktan sonra, konuşma akışını yönetir:

- **Sistem Yönergesi (System Prompt):** Chatbot, kimliğini görevini ve kurallarını tanımlayan çok detaylı bir sistem yönergesi ile başlatılır. Bu yönerge, modelin hangi durumda hangi aracı seçeceğine dair stratejiler içerir.
- **Fonksiyon Çağırma (Function Calling):** Modele, `get_weather`, `analyze_soil`, `query_organic_farming_knowledge`, `ml_crop_recommendation` ve `research_agent_query` gibi projenin tüm yeteneklerini temsil eden bir "araç" listesi sunulur.
- **Karar Verme:** Kullanıcı "Ankara'da organik domates yetiştirmek için toprak ve hava durumu önerilerin neler?" gibi karmaşık bir soru sorduğunda, `chatbot.py` içindeki model bu soruyu tek bir araçla çözemeyeceğini anlar. Bu noktada, bu karmaşık görevi çözmesi için daha yetenekli bir alt sistemi, yani **Araştırma Ajanı**'nı (`research_agent_query`) tetikler.

LLM Araç Kutusu: Araçlar, Zincirler ve Ajanlar (Tools, Chains & Agents)

Yapay zeka katmanı (LLM), bir görevi yerine getirmek için `chatbot.py` içinde tanımlanmış bir "araç kutusuna" erişir. LLM, kullanıcının talebinin karmaşıklığına bağlı olarak bu araç kutusundan uygun seviyede bir yetenek seçer. Bu yetenekler basit (tekil araçlar), orta (zincirler) veya gelişmiş (ajanlar) olabilir.

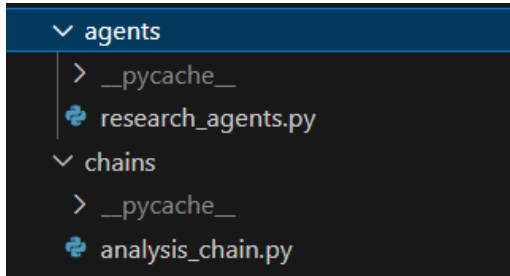
1. Tekil Araçlar (Individual Tools)



Kullanıcıdan "Hava durumu nasıl?" veya "Organik tarım nedir?" gibi net ve tek bir cevap gerektiren talepler geldiğinde, LLM orkestratörü (`chatbot.py`) doğrudan ilgili aracı çağırır:

- **get_weather:** `weather_tool.py` dosyasını kullanarak Backend API'den (Port 8000) gerçek zamanlı hava durumu verilerini (günlük/saatlik) çeker.
- **analyze_soil / get_automatic_location_soil:** `soil_analyzer_tool.py` aracılığıyla, Backend API'den alınan toprak verilerini analiz eder ve kullanıcıya bir rapor formatında sunar.
- **query_organic_farming_knowledge:** RAG katmanını tetikler. `rag_tool.py` aracılığıyla ChromaDB vektör veritabanında anlamsal bir arama yapar ve bulunan doküman parçacıklarını LLM'e (Gemini) göndererek kaynaklara dayalı bir cevap oluşturur.
- **ml_crop_recommendation:** `ml_tool.py` aracılığıyla Makine Öğrenmesi API'sini (Port 8003) çağırır. Bu araç, toprak ve iklim verilerini birleştirerek en uygun ürün tavsiyelerini olasılık skorlarıyla birlikte döndürür.
- **Diğer Yardımcı Araçlar:** Sistem ayrıca `data_visualitor_tool.py` gibi, ham JSON verisini özetleyen veya metin raporuna dönüştüren iç araçlara da sahiptir.

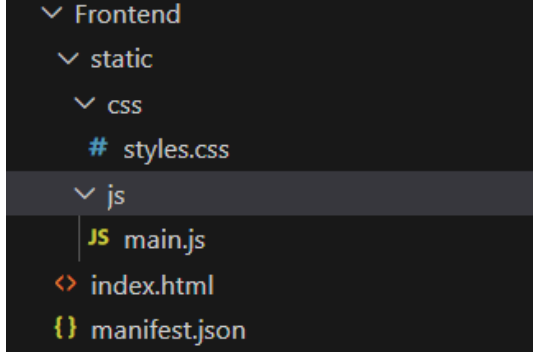
2. Orta ve Gelişmiş İş Akışları (Chains & Agents)



Kullanıcı talebi basit bir "araç" çağrısından daha karmaşık olduğunda, LLM katmanı bu tekil araçları kullanan önceden tanımlanmış daha güçlü iş akışlarını devreye sokar:

- **Analiz Zinciri (analysis_chain.py)**
 - Bu, Ajandan daha basit, önceden tanımlanmış sıralı bir iş akışıdır. chatbot.py tarafından comprehensive_soil_analysis fonksiyonu olarak çağrılabilir.
 - Bir "zincir" olarak, belirli bir sırayı takip eder:
 1. Veriyi al.
 2. DataVisualizerTool ile veriyi özetle
 3. SoilAnalyzerTool ile toprağı analiz et.
 4. Bulguları birleştirerek nihai bir rapor oluştur.
- **Araştırma Ajanı (research_agents.py)**
 - Bu, projenizdeki en gelişmiş iş mantığıdır. chatbot.py tarafından research_agent_query olarak tetiklendiğinde, bu ajan kendi "beynine" sahip bir alt-LLM olarak çalışır.
 - Kendi araç setine sahiptir (WeatherTool, SoilAnalyzerTool, RAGTool, CropRecommendationTool).
 - Kullanıcının karmaşık sorgusunu (örn: "Ankara'da domates...") alır, bu sorguyu alt adımlara böler:
 - "Önce Ankara için hava durumu verisini çekmeliyim." (WeatherTool'u çağırır).
 - "Sonra 'domates' ve 'organik' kelimeleri için RAG veritabanını sorgulamalıyım." (RAGTool'u çağırır).
 - "Eğer koordinat verilmişse, toprak analizini de yapmalıyım." (SoilAnalyzerTool'u çağırır).
 - Tüm bu araçlardan topladığı farklı bilgileri birleştirir, bir sentez yapar ve kullanıcıya kapsamlı, tek bir araştırma raporu sunar.

2.4. Frontend Katmanı (Kullanıcı Arayüzü)



Bu katman, UMay sisteminin kullanıcı ile etkileşime girdiği görsel arayüzdür. Modern web teknolojileri kullanılarak geliştirilmiş olup, kullanıcının sistemi kolayca kullanabilmesini ve sunulan bilgileri (hava durumu, toprak analizi, chatbot cevapları) rahatça takip edebilmesini sağlar.

- **Teknoloji:** Temel olarak **HTML5, CSS3 ve JavaScript** kullanılarak geliştirilmiştir. Herhangi bir büyük JavaScript framework'üne (React, Vue, Angular gibi) bağımlılığı bulunmamaktadır.
- **Sunucu:** Frontend dosyaları (index.html, styles.css, main.js), ana orkestratör olan main_chatbot.py içindeki **FastAPI** sunucusu tarafından **Port 8001** üzerinden sunulur. Bu sayede, Frontend ve LLM Backend aynı port üzerinden haberleşebilir.

2.4.1. Arayüz Bileşenleri ve Özellikleri

Kullanıcı arayüzü (index.html ve styles.css ile tanımlanmıştır), aşağıdaki ana bileşenlerden oluşur:

1. **Merkezi Chat Paneli:** Kullanıcının UMay ile etkileşime girdiği ana sohbet penceresidir.
 - Kullanıcı mesajları ve UMay'nın yanıtları burada görüntülenir.
 - Mesajlar, gönderen (kullanıcı/bot) ve zaman damgasıyla birlikte gösterilir.
 - Yapay zeka yanıt verirken bir "yazıyor..." (typing) göstergesi bulunur.
 - Sohbet geçmişi otomatik olarak tarayıcının yerel depolama alanına (localStorage) kaydedilir.
2. **Yan Bilgi Panelleri (Hava Durumu ve Toprak Analizi):** Chat panelinin sağında ve solunda yer alan bu paneller, kullanıcının bulunduğu konum için anlık hava durumu ve toprak analizi özetlerini gösterir.
 - Bu panellerdeki veriler, Frontend'in main_chatbot.py üzerindeki /weather/ ve /soil/ endpoint'lerine (Port 8001) istek atmasıyla alınır. Bu istekler daha sonra main_chatbot.py tarafından ilgili Backend API servislerine (Port 8000) yönlendirilir.

3. **Sohbet Geçmişi Kenar Çubuğu (Sidebar):** Kullanıcının önceki sohbetlerini görmesini, silmesini veya kaldığı yerden devam etmesini sağlayan açılır/kapanır bir menüdür.
4. **Tema Değiştirme:** Kullanıcının açık ve koyu tema arasında geçiş yapmasını sağlar.
5. **Duyarlı Tasarım (Responsive Design):** Arayüz, farklı ekran boyutlarına (masaüstü, tablet, mobil) uyum sağlayacak şekilde tasarlanmıştır.

2.4.2. Backend ile Etkileşim

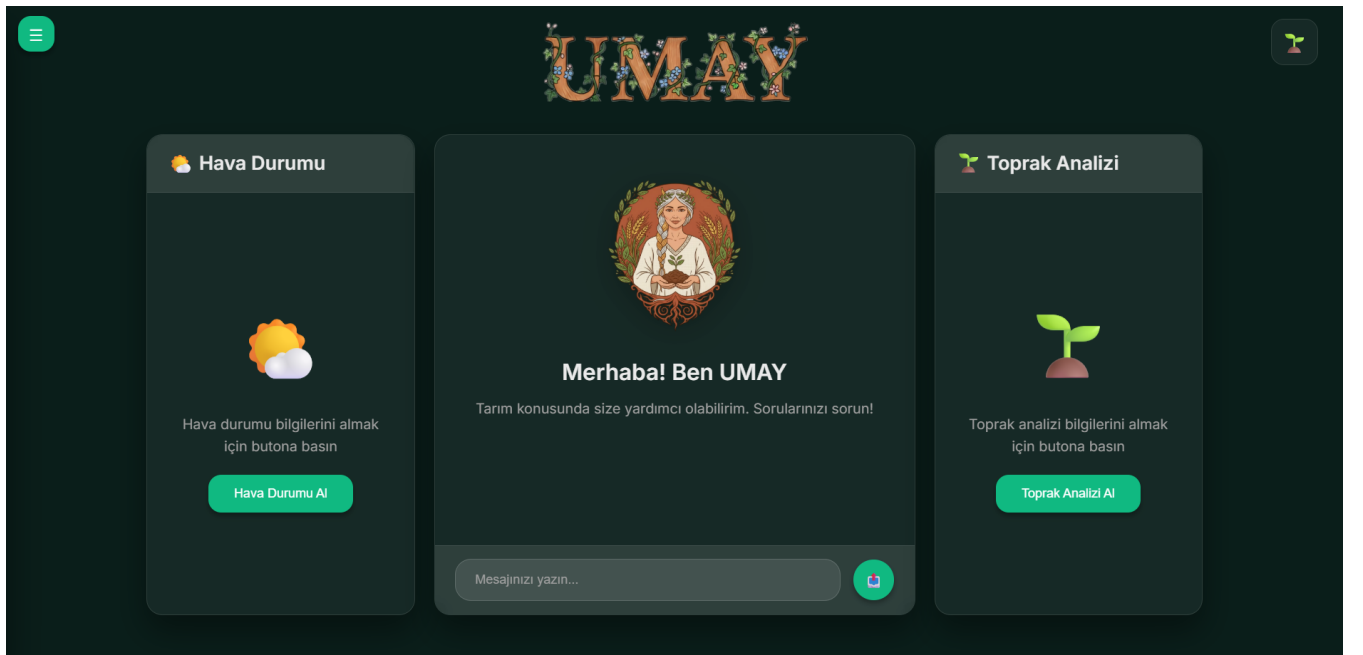
Frontend (main.js), Backend ile tamamen HTTP istekleri üzerinden iletişim kurar:

Kullanıcı sohbet penceresine bir mesaj yazdığında, main.js bu mesajı JSON formatında paketlenerek <http://localhost:8001/chat/> adresine bir POST isteği gönderir.

- main_chatbot.py, bu isteği alır, chatbot.py aracılığıyla işler (gerekirse diğer API'leri veya RAG'ı çağırır) ve üretilen cevabı yine JSON formatında Frontend'e geri döndürür.
- Benzer şekilde, yan panellerdeki "Hava Durumu AI" veya "Toprak Analizi AI" butonlarına tıklandığında, main.js sırasıyla /weather/ ve /soil/ endpoint'lerine istek atar ve dönen verileri ilgili panellerde gösterir.
- Frontend, tarayıcının Geolocation API'sini kullanarak kullanıcının mevcut konumunu alabilir ve bu bilgiyi Backend'e (otomatik analizler için) gönderebilir.

2.4.4 Frontend Yapısı Görsellerle Örnek:

2.4.4.1 Genel Görünüm



2.4.4.2 Örnek Senaryolar

Örnek Senaryo 1

Kullanıcı hava durumu bilgisini hem sol taraftaki panelden öğrenebilir hem de chatbota sorarak öğrenebilir.



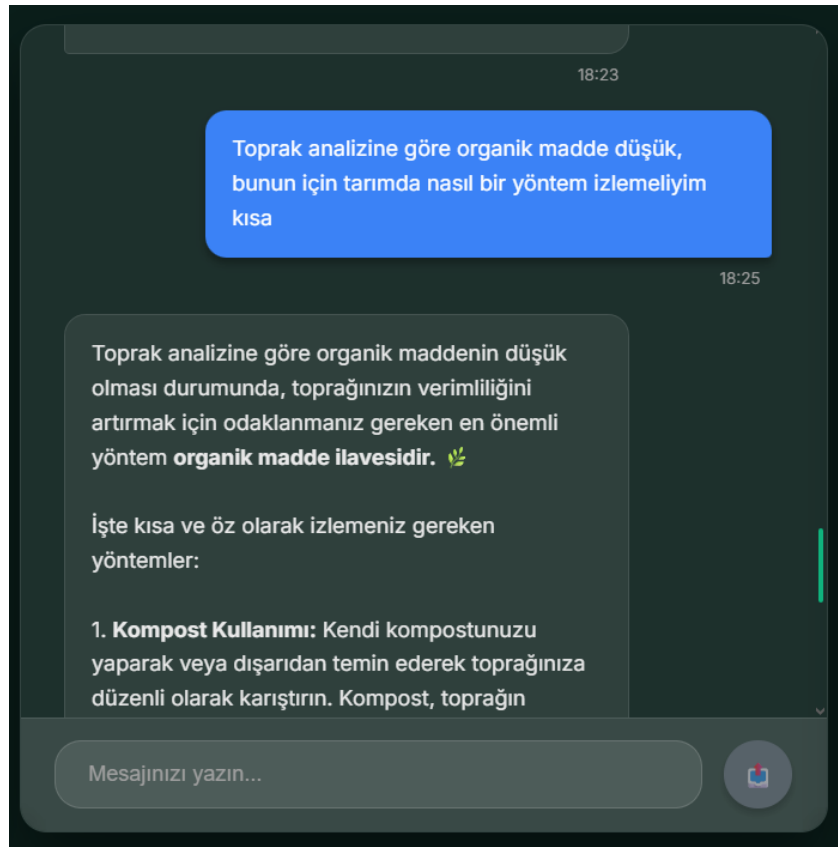
Örnek Senaryo 2

Kullanıcı hem sağ taraftaki panelden bulunduğu konumun toprak yapısını öğrenebilir hem de spesifik bir koordinatın veya bir şehrin toprak yapısını öğrenebilir.



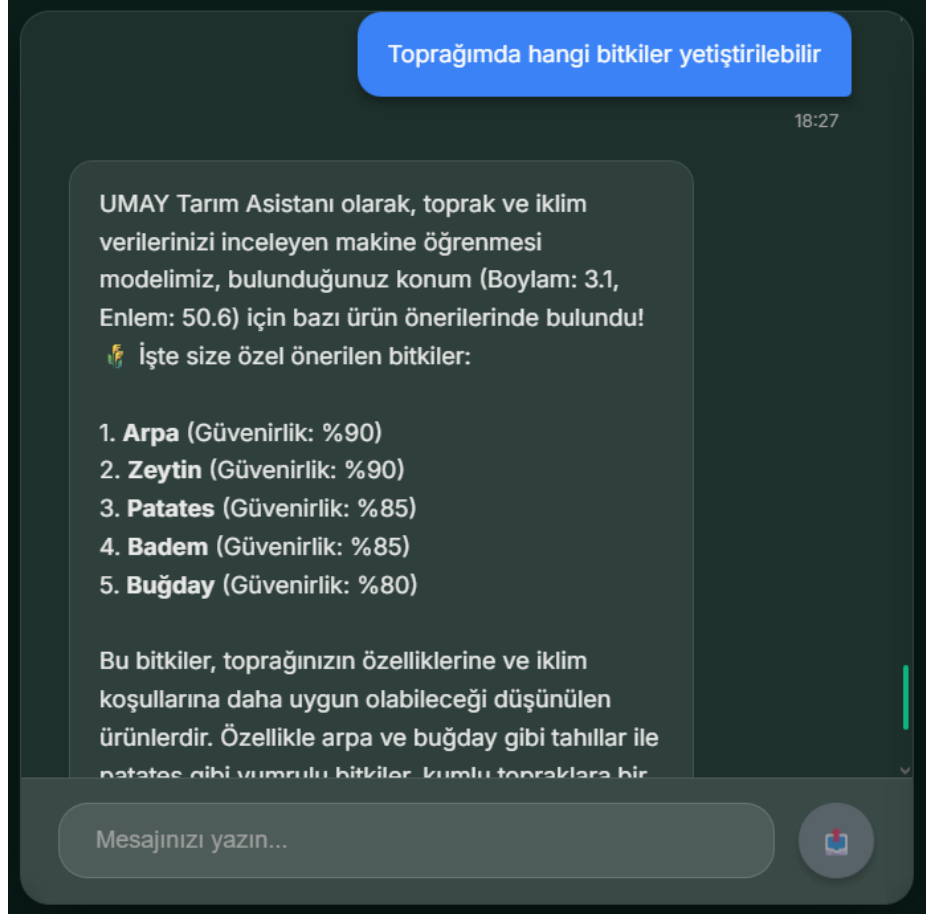
Örnek Senaryo 3

Kullanıcı teknik bilgiler sorabilir sorunlarını chatbota iletebilir ve çözüm tavsiyeleri alabilir.



Örnek Senaryo 4

Kullanıcı belirli bir bölgede hangi tarımsal ürünleri yetiştirmesi gerektiğini öğrenebilir.



Sonuç olarak, UMay'nın sistem mimarisi, modern yapay zeka tekniklerini (LLM, RAG, ML) geleneksel veri servisleriyle (API'ler, veritabanları) akıllıca birleştiren modüler ve güçlü bir yapı sunmaktadır.

LLM katmanı, sistemin beyni olarak, kullanıcının doğal dil talebini anlamlandırır ve görevin karmaşıklığına göre en uygun aracı, zinciri veya ajanı seçerek harekete geçer. Backend API katmanı, toprak ve hava durumu gibi nicel verileri sağlarken, RAG katmanı organik tarım gibi konularda derinlemesine nitel bilgi sunar.

LLM, bu farklı kaynaklardan gelen bilgileri sentezleyerek, araçların(toolların) tek başına sağlayamayacağı bütüncül ve bağlama uygun cevaplar üretir. Bu entegre yaklaşım, UMay'nın sadece bir bilgi sağlayıcı değil, aynı zamanda veriye dayalı, eyleme geçirilebilir tavsiyeler sunabilen akıllı bir tarım asistanı olma vizyonunu gerçekleştirmesini sağlar. Son olarak, tüm bu karmaşık işlemlerin sonucu, Frontend katmanı aracılığıyla kullanıcıya anlaşılır ve etkileşimli bir arayüzle sunulur.

İleride Yapılması Planlananlar

Projenin mevcut güçlü altyapısını daha da ileriye taşımak ve kullanıcı deneyimini zenginleştirmek amacıyla gelecek dönemde odaklanılması planlanan temel geliştirme alanları şunlardır:

- **Mobil Uyumluluk:** Frontend katmanında halihazırda bulunan duyarlı tasarımın iyileştirilmesi planlanmaktadır. Hedef, UMay arayüzünün tablet ve akıllı telefonlar gibi tüm mobil cihazlarda daha akıcı ve tam uyumlu bir kullanıcı deneyimi sunmasıdır.
-
- **Bildirim Sistemi:** Kullanıcılara kritik tarımsal uyarılar gönderebilecek proaktif bir bildirim sistemi geliştirilmesi hedeflenmektedir. Bu sistem, UMay Asistan'ın tespit ettiği (örneğin, hava durumu servisinden alınan veriye göre) ani don tehlikesi, şiddetli hava olayları veya RAG bilgi bankasındaki bilgilere dayalı zararlı böcek uyarıları gibi acil durumlar için anlık bildirimler sağlayacaktır.
-
- **Docker Entegrasyonu:** Projenin kurulumunu, dağıtımını (deployment) ve ölçeklendirilmesini basitleştirmek amacıyla Docker konteynerizasyonu planlanmaktadır. Bu sayede, Backend API (Soil, Weather), ML API ve ana LLM orkestratörü gibi tüm modüler servisler, bağımsız ve taşınabilir konteynerler halinde paketlenerek farklı ortamlarda tutarlı bir şekilde çalışması sağlanacaktır.